

REPORTE DE RETROSPECTIVA

EcoRoute — Lecciones Aprendidas y Acciones de Mejora

INTRODUCCIÓN

Este documento registra los reportes de retrospectiva de cada sprint del proyecto EcoRoute. Las retrospectivas siguen el formato estándar "**Qué salió bien / Qué salió mal / Qué mejorar**" complementado con acciones concretas, responsables y fechas de seguimiento. El objetivo es mejorar continuamente el proceso de desarrollo a lo largo del proyecto.

RETROSPECTIVA — SPRINT 1

Período: Semanas 1–5

Participantes: Equipo completo

Facilitador: Líder del proyecto

Fecha: Por definir

1. Qué salió bien ✓

ID	Aspecto positivo	Impacto
B-S1-01	La definición de los requisitos funcionales y no funcionales estuvo completamente lista antes de iniciar el sprint, lo que evitó reprocesos durante la implementación	El equipo comenzó el Sprint 2 con claridad total sobre qué construir
B-S1-02	El archivo config.yaml centralizado fue bien recibido por todo el equipo desde el inicio y se adoptó sin resistencia	Cero parámetros hardcodeados en el código del sprint
B-S1-03	La instalación en Linux siguió exactamente el README sin pasos adicionales	README validado como suficiente para nuevos miembros
B-S1-04	Los acuerdos DoR y DoD fueron discutidos y aceptados por consenso en la primera reunión del equipo	Criterios claros desde el día 1 sin discusiones posteriores

2. Qué salió mal ✗

ID	Problema identificado	Impacto	Causa raíz
P-S1-01	La instalación en Windows requirió pasos adicionales no documentados para configurar las variables de entorno de SUMO	HU-26 no pudo marcarse como Done al cierre del sprint	Falta de prueba de instalación en Windows antes de documentar el README

P-S1-02	La validación de tipos en ConfigLoader fue más compleja de lo estimado porque pydantic requirió configuración adicional no prevista	HU-02 tomó 1 día más de lo planificado	Story points subestimados: se asignaron 2 pero el esfuerzo real fue 3
P-S1-03	Los Pull Requests tardaron más de 24 horas en ser revisados en dos ocasiones, bloqueando el avance	Un miembro del equipo esperó 36 horas para hacer merge	El equipo no tenía un canal claro de notificación de PRs abiertos

3. Qué mejorar y acciones concretas

ID	Acción de mejora	Responsable	Fecha límite	Estado
A-S1-01	Agregar sección de instalación en Windows al README con las variables de entorno de SUMO documentadas paso a paso. Probar la instalación en una máquina Windows limpia antes de cerrar la acción.	Desarrollador	Semana 1 del Sprint 2	Pendiente
A-S1-02	En el próximo refinamiento de backlog usar Planning Poker con mayor rigor para historias de validación y configuración. Agregar al DoR que historias con dependencias de bibliotecas nuevas (pydantic, etc.) deben incluir un spike de estimación.	Líder del proyecto	Sprint Planning Sprint 2	Pendiente
A-S1-03	Crear canal de notificación de PRs abiertos (mensaje en grupo del equipo o etiqueta en Git). Reafirmar el acuerdo de revisión en 24 horas en el próximo daily.	Líder del proyecto	Día 1 del Sprint 2	Pendiente

4. Métricas del Sprint

Métrica	Valor planificado	Valor real
Story Points comprometidos	12	12
Story Points completados (Done)	12	10
Historias completadas	5	4 (HU-26 pasa a Sprint 2)
Velocidad del equipo	—	10 SP
Deuda técnica resuelta	3 ítems (DT-04, DT-14, DT-15)	Por confirmar
Pull Requests abiertos	—	6

Pull Requests con revisión > 24h	0	2
----------------------------------	---	---

5. Acuerdos modificados en esta retrospectiva

Acuerdo modificado	Cambio
DoR-12	Se agrega: "Si la historia introduce una biblioteca nueva (no listada en environment.yml), debe incluir un spike de estimación de 2 horas antes de asignar story points definitivos"

RETROSPECTIVA — SPRINT 2

Período: Semanas 6–10

Participantes: Equipo completo

Facilitador: Por definir

Fecha: Por definir

1. Qué salió bien ✓

ID	Aspecto positivo	Impacto
B-S2-01	La abstracción del entorno con interfaz Gymnasium facilitó enormemente las pruebas de integración	Las pruebas de integración del módulo de entorno se escribieron en la mitad del tiempo estimado
B-S2-02	El módulo de emisiones con modelos intercambiables (VSP/COPERT) fue implementado sin necesidad de modificar otros módulos	Cumplimiento del principio de configuración centralizada sin excepciones
B-S2-03	El canal de notificación de PRs implementado tras el Sprint 1 redujo los tiempos de revisión	0 PRs tardaron más de 24 horas en ser revisados

2. Qué salió mal X

ID	Problema identificado	Impacto	Causa raíz
P-S2-01	La integración con SUMO vía TraCI fue más compleja de lo esperado — el proceso SUMO debía iniciarse manualmente antes de ejecutar las pruebas	Las pruebas de integración fallaban en entornos de CI sin SUMO corriendo	Falta de automatización del inicio de SUMO en el entorno de pruebas

P-S2-02	La normalización del vector de estado presentó valores fuera del rango [0,1] para escenarios de alta densidad no contemplados en el diseño inicial	HU-07 requirió revisión y corrección tras la primera integración	Los valores máximos del estado no estaban bien definidos en el SRS
P-S2-03	La función de recompensa sin normalización de componentes (DT-05) produjo comportamientos de entrenamiento inesperados que detectamos tarde	Se identificó como deuda técnica prioritaria para el Sprint 3	El impacto de DT-05 fue subestimado en la planificación

3. Qué mejorar y acciones concretas

ID	Acción de mejora	Responsable	Fecha límite	Estado
A-S2-01	Implementar fixture de pytest en confest.py que inicie y cierre el proceso SUMO automáticamente antes y después de las pruebas de integración. Documentar en el README cómo ejecutar las pruebas con SUMO.	Desarrollador	Semana 1 del Sprint 3	Pendiente
A-S2-02	Agregar al SRS una tabla de rangos máximos esperados para cada componente del vector de estado. Actualizar StateBuilder para manejar saturación de valores con clipping entre 0 y 1.	Analista + Desarrollador	Sprint Planning Sprint 3	Pendiente
A-S2-03	Subir la prioridad de DT-05 (normalización de recompensa) al inicio del Sprint 3. Agregar al DoD que historias que modifiquen la función de recompensa requieren prueba de convergencia de 50 episodios.	Líder del proyecto	Sprint Planning Sprint 3	Pendiente

4. Métricas del Sprint

Métrica	Valor planificado	Valor real
Story Points comprometidos	26 (24 + 2 de HU-26)	26
Story Points completados (Done)	26	22
Historias completadas	9	8 (HU-07 requirió revisión)
Velocidad del equipo	10 SP (Sprint 1)	22 SP

Bugs detectados en sprint	—	2
Deuda técnica nueva identificada	—	DT-05 subida a prioridad Alta

5. Acuerdos modificados en esta retrospectiva

Acuerdo modificado	Cambio
DoD-10	Se agrega: "Las pruebas de integración que involucren SumoEnv deben usar el fixture de confstest.py que gestiona automáticamente el proceso SUMO"
DoD-12	Se agrega: "Historias que modifiquen la función de recompensa requieren prueba de convergencia de mínimo 50 episodios antes de marcarse como Done"

RETROSPECTIVA — SPRINT 3

Período: Semanas 11–14

Participantes: Equipo completo

Facilitador: Por definir

Fecha: Por definir

1. Qué salió bien ✓

ID	Aspecto positivo	Impacto
B-S3-01	La implementación del agente DQN con target network y replay buffer fue más fluida de lo esperado gracias a la documentación del ADR-007	El módulo del agente fue completado con 2 días de anticipación
B-S3-02	La resolución de DT-05 (normalización de recompensa) al inicio del sprint evitó problemas de convergencia en las pruebas de integración del Trainer	El entrenamiento de prueba de 100 episodios mostró convergencia clara
B-S3-03	El fixture de SUMO implementado en Sprint 2 funcionó perfectamente en todas las pruebas de integración del Sprint 3	0 fallos de pruebas por problemas de conexión SUMO

2. Qué salió mal ✗

ID	Problema identificado	Impacto	Causa raíz
P-S3-01	La prueba de convergencia de 100 episodios requerida por el DoD tomó más tiempo del esperado en hardware sin GPU	El cierre de algunas historias del Sprint 3 se retrasó por espera de resultados	Hardware de desarrollo sin GPU subestimado en la planificación

P-S3-02	Los controladores baseline generaron métricas con escenarios ligeramente distintos al DQN por un bug en la semilla de SUMO	La comparación del Sprint 3 fue inválida estadísticamente y debió repetirse	Bug en la asignación de semilla al proceso SUMO en baselines.py
P-S3-03	El decaimiento de epsilon no estaba siendo aplicado correctamente en las primeras 20 iteraciones por un error de inicialización	El agente exploró menos de lo esperado en los primeros episodios	Error detectado tarde en la revisión de código

3. Qué mejorar y acciones concretas

ID	Acción de mejora	Responsable	Fecha límite	Estado
A-S3-01	Explorar acceso a Google Colab para las pruebas de convergencia largas. Agregar al DoR que historias del agente DQN que requieran prueba de convergencia deben planificarse con acceso a GPU.	Investigador	Sprint Planning Sprint 4	Pendiente
A-S3-02	Agregar prueba automatizada que verifica que DQN y baselines usan exactamente la misma semilla de SUMO. Incluir en el DoD de historias de baselines.	Desarrollador	Semana 1 del Sprint 4	Pendiente
A-S3-03	Agregar al checklist de code review la verificación explícita del decaimiento de epsilon en historias que modifiquen la política del agente.	Líder del proyecto	Inmediato	Pendiente

4. Métricas del Sprint

Métrica	Valor planificado	Valor real
Story Points comprometidos	25	25
Story Points completados (Done)	25	21
Historias completadas	7	6 (HU-18 pasa a Sprint 4)
Velocidad del equipo	22 SP (Sprint 2)	21 SP
Bugs detectados en sprint	—	2 (P-S3-02, P-S3-03)

Bugs resueltos en sprint	—	2
Deuda técnica resuelta	DT-01, DT-02, DT-03, DT-05, DT-07	Por confirmar

5. Acuerdos modificados en esta retrospectiva

Acuerdo modificado	Cambio
DoD — Checklist code review	Se agrega ítem: "Si la historia modifica la política epsilon-greedy, verificar explícitamente que el decaimiento es correcto desde el episodio 1"
DoR-11	Se agrega: "Historias de baselines deben especificar explícitamente la semilla de SUMO a usar para garantizar comparabilidad con el DQN"

RETROSPECTIVA — SPRINT 4

Período: Semanas 15–20

Participantes: Equipo completo

Facilitador: Por definir

Fecha: Por definir

1. Qué salió bien ✓

ID	Aspecto positivo	Impacto
B-S4-01	El módulo de evaluación con Logger y Visualizer fue implementado de forma modular permitiendo agregar nuevas gráficas sin modificar el logger	Extensibilidad del módulo validada en la práctica
B-S4-02	Los resultados experimentales mostraron reducción de emisiones CO2 del agente DQN superior al criterio de aceptación del 10% vs baseline de tiempo fijo	Hipótesis central del proyecto validada
B-S4-03	La prueba de significancia estadística (Mann-Whitney U) fue integrada directamente en el Visualizer, eliminando el paso manual de cálculo en herramientas externas	Rigor científico automatizado en el flujo de evaluación

2. Qué salió mal ✗

ID	Problema identificado	Impacto	Causa raíz
P-S4-01	La generación de gráficas con matplotlib bloqueó el hilo principal durante evaluaciones largas (DT-	Los experimentos de evaluación tardaron más de lo esperado	DT-08 fue postergado a Iteración 3 y no fue

	08 identificado previamente pero no resuelto)		resuelto antes del Sprint 4
P-S4-02	El reporte de cobertura final mostró el módulo visualizer.py con 68%, por debajo del mínimo aceptable del 70%	Criterio DoD-R02 no cumplido en primera revisión	Las ramas de exportación PDF con backend no configurado no estaban cubiertas
P-S4-03	La documentación del README no cubría el flujo de evaluación con checkpoints, solo el de entrenamiento	Nuevos investigadores no sabían cómo evaluar el modelo sin consultar el código	El README fue escrito al inicio y no se actualizó con los nuevos comandos del Sprint 4

3. Qué mejorar y acciones concretas

ID	Acción de mejora	Responsable	Fecha límite	Estado
A-S4-01	Resolver DT-08 (visualización en hilo separado) como primera tarea de mantenimiento post-Sprint 4. Agregar al DoD que historias de visualización no pueden cerrarse con DT-08 abierto.	Desarrollador	Post-Sprint 4	Pendiente
A-S4-02	Agregar configuración de backend no interactivo (matplotlib Agg) en conftest.py para cubrir ramas de exportación PDF. Meta: visualizer.py $\geq$ 75% de cobertura.	Desarrollador	Post-Sprint 4	Pendiente
A-S4-03	Actualizar README con sección de evaluación de modelos: cómo cargar un checkpoint, ejecutar evaluación y generar gráficas comparativas. Agregar al DoD que el README debe actualizarse cuando se agrega un comando nuevo al sistema.	Desarrollador	Post-Sprint 4	Pendiente

4. Métricas del Sprint

Métrica	Valor planificado	Valor real
Story Points comprometidos	15 (14 + HU-18)	15
Story Points completados (Done)	15	15
Historias completadas	7	7
Velocidad del equipo	21 SP (Sprint 3)	15 SP

Cobertura global del proyecto	$\geq 80\%$	Por confirmar
Reducción de emisiones DQN vs tiempo fijo	$\geq 10\%$	Por confirmar
p-value comparación estadística	< 0.05	Por confirmar
Deuda técnica resuelta	DT-06, DT-08, DT-09, DT-10	DT-06, DT-09, DT-10 resueltos. DT-08 pendiente

5. Acuerdos modificados en esta retrospectiva

Acuerdo modificado	Cambio
DoD-15	Se agrega: "Si la historia agrega un nuevo comando de ejecución al sistema, el README debe actualizarse antes de marcarla como Done"
DoD-R02	Se agrega: "El módulo visualizer.py debe alcanzar $\geq 75\%$ de cobertura con backend matplotlib Agg configurado en confest.py"

RESUMEN DE LECCIONES APRENDIDAS — PROYECTO COMPLETO

Lo que funcionó bien y debe replicarse

Lección	Aplicación futura
La configuración centralizada en YAML desde el día 1 eliminó casi completamente los parámetros hardcodeados	Adoptar este patrón como requisito desde el inicio en proyectos similares
Los ADR documentados antes de implementar evitaban discusiones técnicas durante el desarrollo	Escribir ADR antes de cada decisión tecnológica mayor, no después
El fixture de pytest para SUMO resolvió definitivamente los problemas de pruebas de integración	En proyectos con simuladores externos, crear el fixture de test desde el Sprint 1
Los criterios DoR impidieron que historias ambiguas entraran al sprint	Mantener el DoR estricto aunque presione la velocidad a corto plazo

Lo que debe mejorar en proyectos futuros

Lección	Acción futura
La deuda técnica DT-05 (normalización de recompensa) debió resolverse en el Sprint 1, no en el Sprint 3	Resolver deuda técnica que afecte el algoritmo principal antes de implementar los módulos que dependen de ella

Las pruebas de convergencia largas no fueron consideradas en la estimación de story points	Incluir tiempo de validación experimental en los story points de historias del agente DQN
El README no se actualizó de forma continua durante el proyecto	Agregar actualización del README como criterio DoD desde el inicio
La cobertura de visualizer.py fue la más baja por no configurar el backend de matplotlib en pruebas	Configurar backends no interactivos para herramientas de visualización desde el primer sprint